

19 | Rust的宏体系：为自己的项目写一个简单的声明宏

唐刚 · Rust 语言从入门到实战



你好，我是 Mike，今天我们一起学习 Rust 语言中有关宏的知识。

宏是一套预处理设施。它的输入是代码本身，对代码进行变换然后输出新的代码。一般来说，输出的新代码必须是合法的当前语言的代码，用来喂给当前语言的编译器进行编译。

宏不是一门语言的必备选项，Java、Go 等语言就没有宏，而 C、CPP、Rust 等语言有宏，而且它们的宏工作方式不一样。

在 Rust 语言中，宏也属于语言的外围功能，用来增强 Rust 语言的核心功能，让 Rust 语言变得更方便好用。宏不属于 Rust 语言的核心，但这并不是说宏在 Rust 中不重要。其实在 Rust 代码中，宏随处可见，掌握宏的原理和用法，有助于我们编写更高效的 Rust 代码。

在 Rust 中，宏的解析和执行是在 Rust 代码的编译阶段之前。你可以理解成，在 Rust 代码编译之前有一个宏展开的过程，这个过程的输出结果就是完整版的 Rust 代码，然后 Rust 编译器再来编译这个输出的代码。

Rust 语言中的宏

当前版本的 Rust 中有两大类宏：声明宏（declarative macro）和过程宏（procedure macro），而过程宏又细分成三种：派生宏、属性宏和函数宏。

下面我们分别介绍一下它们。

声明宏

声明宏是用 `macro_rules!` 定义的宏，我们常见的 `println!()`、`vec![]` 都是这种宏。比如 `vec![]` 是按类似下面这种方式定义的：

[复制代码](#)

```
1 #[macro_export]
2 macro_rules! vec {
3     ( $( $x:expr ),* ) => {
4         {
5             let mut temp_vec = Vec::new();
6             $(
7                 temp_vec.push($x);
8             )*
9             temp_vec
10        }
11    };
12 }
```

注：代码来自官方 The Book，这是一个演示版的定义，实际的 `vec!` 的定义比这个要复杂得多。

这是什么代码？完全看不懂，不是正常的 Rust 代码吧？是的，为了对 Rust 代码本身进行操作，需要重新定义一套语法形式，而这种语法形式，只在 `macro_rules!` 里有效。

宏整体上来说，使用了一种**代码匹配 + 生成机制**来生成新的代码。上面代码里的 `$()` 用来匹配代码条目。`$x:expr` 表示匹配的是一个表达式，匹配后的条目用 `$x` 代替。`*` 表示前面这个模式可以重复 0 次或者 1 次以上，这个模式就是 `$( $x:expr ),`，注意 `$()` 后面有个 `,` 号，这个逗号也是这个模式中的一部分，在匹配的时候是一个可选项，有就匹配，遇到最

后一个匹配项的时候，就忽略它。=> 前面是起匹配作用的部分，=> 后面是生成代码的部分。

在生成代码的部分中，\$() 号和 => 前面那个 \$() 的作用差不多，就是表明被包起来的这块代码是可重复的。紧跟的 * 表示这个代码块可以重复 0 次到多次。具体次数等于 => 号前面的 * 号所代表的次数，两个一致。下面我们创建一个拥有三个整数元素的 Vec。

[复制代码](#)

```
1 let v = vec![1,2,3];
```

代码展开后实际就是：

[复制代码](#)

```
1 {  
2     let mut temp_vec = Vec::new();  
3     temp_vec.push(1);  
4     temp_vec.push(2);  
5     temp_vec.push(3);  
6     temp_vec  
7 }
```

你可以看到，temp_vec.push(); 重复了 3 次，这是因为 1,2,3 匹配 \$x: expr，匹配了 3 次，分别匹配出 1、2、3 三个整数。然后就生成了三行 temp_vec.push();。

你可能会问，vec![] 这对中括号哪里去了？其实在 Rust 中，你用 ()、[]、{} 都可以，比如：

[复制代码](#)

```
1 let a = vec!(0, 1, 2);  
2 let b = vec![0, 1, 2];  
3 let c = vec! { 0, 1, 2 };
```

不过也确实存在一些习惯性的表述，比如对于 Vec 这种列表，用 [] 显得更地道，其他语言也多用 []，这样能够和程序员的习惯保持一致。对于类函数式的调用，使用 () 更地道。对于构建结构体之类的宏或者存在大段代码输入的情况，用 {} 更合适。

现在你是不是能读懂一点了？

上面代码里的 expr 表示要匹配的 item 是个表达式。常见的匹配方式有 7 种。

expr: 匹配表达式

ty: 匹配类型

stmt: 匹配语句

item: 匹配一个 item

ident: 匹配一个标识符

path: 匹配一个 path

tt: 匹配一个 token tree

完整的说明，你可以看我给出的参考 [🔗 链接](#)。

常见的重复符号有 3 个。

* 表示重复 0 到多次。

+ 表示重复 1 到多次。

? 表示重复 0 次或 1 次。

利用刚刚学到的知识，我们可以自己动手写一个声明宏。

自己动手写一个声明宏

目标：实现一个加法宏 `add!(1,2)`，输出结果 3。

你可以看一下示例代码。

[📄 复制代码](#)

```
1 macro_rules! add {
2     // 第一个分支，匹配两个元素的加法
3     ($a:expr, $b:expr) => {
4         {
5             $a+$b
6         }
7     };
8     // 第二个分支：当只有一个元素时，也应该处理，这是边界情况
9     ($a:expr) => {
10        {
11            $a
12        }
13    }
14 }
15
16 fn main(){
17     let x=0;
18     let sum = add!(1,2); // 调用宏
19     let sum = add!(x);
20 }
```

通过示例我们可以看到，声明宏里面可以写多个匹配分支，Rust 会根据匹配到的模式自动选择适配的分支进行套用。

我们可以用 `cargo expand` 来展开这个代码。

The screenshot shows the Rust Playground interface. The top bar has buttons for RUN, DEBUG, STABLE, and a menu icon. On the right, there are buttons for SHARE, TOOLS, CONFIG, and a help icon. The main editor area contains the following Rust code:

```
1 macro_rules! add {  
2     // 第一个分支，匹配两个元素的加法  
3     ($a:expr, $b:expr) => {  
4         {  
5             $a+$b  
6         }  
7     };  
8     // 第二个分支：当只有一个元素时，也应该处理，这是边界情况  
9     ($a:expr) => {  
10        {  
11            $a  
12        }  
13    }  
14 }  
15
```

Below the editor, there are three tabs: Execution, Standard Error, and Macro expansion. The Execution tab is active, showing the following output:

```
Compiling playground v0.0.1 (/playground)  
Finished dev [unoptimized + debuginfo] target(s) in 0.40s  
  
#![feature(prelude_import)]  
#[prelude_import]  
use std::prelude::rust_2021::*;  
#[macro_use]  
extern crate std;  
macro_rules! add {  
    ($a : expr, $b : expr) =>  
    {  
        {  
            $a + $b  
            // 第一个分支，匹配两个元素的加法  
        }  
    }  
}; ($a : expr) =>  
{  
    {  
        $a  
        // 第二个分支：当只有一个元素时，也应该处理，这是边界情况  
    }  
}
```

On the right side, there is a TOOLS panel with the following options:

- Rustfmt**: Format this code with Rustfmt. 1.7.0-nightly (2023-11-13 ba7c7a3)
- Clippy**: Catch common mistakes and improve the code using the Clippy linter. 0.1.75 (2023-11-13 ba7c7a3)
- Miri**: Execute this program in the Miri interpreter to detect certain cases of undefined behavior (like out-of-bounds memory access). 0.1.0 (2023-11-13 ba7c7a3)
- Expand macros**: Expand macros in code using the nightly compiler. 1.76.0-nightly (2023-11-13 ba7c7a301984967c8c13)

展开后实际是下面这个样子。

复制代码

```
1  #![feature(prelude_import)]  
2  #[prelude_import]  
3  use std::prelude::rust_2021::*;  
4  #[macro_use]  
5  extern crate std;  
6  macro_rules! add {  
7      ($a : expr, $b : expr) =>  
8      {  
9          {  
10             $a + $b  
11             // 第一个分支，匹配两个元素的加法  
12         }  
13     }; ($a : expr) =>  
14     {  
15         {  
16             $a  
17             // 第二个分支：当只有一个元素时，也应该处理，这是边界情况  
18         }  
19     }  
20 }  
21 fn main() {  
22     let x = 0;  
23     let sum = { 1 + 2 }; // 调用宏
```

```

24     let sum = { x };
25 }

```

请你仔细对比展开前后的代码，好好理解一下。不过，仅仅是两个元素的相加，写个宏好像多此一举了。下面我们把它扩展到多个数字的加法。

[复制代码](#)

```

1 macro_rules! add {
2     ( $($a:expr),* ) => {
3         {
4             // 开头要有个0，处理没有任何参数传入的情况
5             0
6             // 重复的块
7             $( + $a )*
8         }
9     }
10 }
11
12 fn main(){
13     let sum = add!();
14     let sum = add!(1,2,3,4);
15 }

```

你可以看一下它展开后的样子。

[复制代码](#)

```

1 #![feature(prelude_import)]
2 #[prelude_import]
3 use std::prelude::rust_2021::*;
4 #[macro_use]
5 extern crate std;
6 macro_rules! add {
7     ( $($a : expr),* ) =>
8     {
9         {
10             0 $(+ $a) *
11             // 开头要有个0，处理没有任何参数传入的情况
12             // 重复的块
13         }
14     }
15 }

```

```
16 fn main() { let sum = { 0 }; let sum = { 0 + 1 + 2 + 3 + 4 }; }
```

可以看到，经过简单的改造，我们的 `add!()` 宏现在能处理无限多的相加项了。如果有时间，你可以进一步考虑如何处理总和溢出的问题。

我们可以先从这种简单的宏入手，边写边展开看效果，实现一个声明宏并不难。

macro_export

在一个模块中写好声明宏后，想要提供给其他模块使用的话，你得使用 `#[macro_export]` 导出，比如：

[复制代码](#)

```
1 mod inner {
2     super::m!();
3     crate::m!();
4 }
5 mod toexport {
6     #[macro_export]    // 请注意这一句，把m!()导出到当前crate root下了
7     macro_rules! m {
8         () => {};
9     }
10 }
11 fn foo() {
12     self::m!(); // main函数在当前crate根下，可这样调用m!()
13     m!();      // 直接调用也是可以的
14 }
```

`#[macro_export]` 可以把你定义的宏导出到当前 crate 根下，这样在 crate 里可以用 `crate::macro_name!` 访问（例子里的第 3 行 `crate::m!()`），在其他 crate 中可以使用 `use crate_name::macro_name` 导入。比如，我们假设上面代码里 crate 的名字是 `mycrate`，在另一个程序里可以这样导入这个宏。

[复制代码](#)

```
1 use mycrate::m;
2
3 fn bar() {
```

```
4     m!();  
5 }
```

macro_use

前面我们使用了 `use mycrate::m` 这种精确的路径来导入 `mycrate` 中的 `m!()` 宏。如果一个 crate 里的宏比较多，我们想一次性全部导入，可以使用 `#[macro_use]` 属性，一次导入一个 crate 中所有已导出的宏，像下面这种写法：

```
1 #[macro_use] extern crate rocket;
```

[复制代码](#)

不过这种写法算是 Rust 早期的遗留写法了，更推荐的还是用到哪个宏就引入哪个宏。有些代码库中还会有这种写法出现，你看到了需要知道是什么意思。

认识过程宏之派生宏

常见的结构体上的 `derive` 标注，就是派生宏。比如我们 [上一节课](#) 讲到的 `thiserror` 提供的 `Error` 派生宏。

```
1 use thiserror::Error;  
2  
3 #[derive(Error, Debug)] // 派生宏  
4 pub enum DataStoreError {  
5     #[error("data store disconnected")] // 属性宏  
6     Disconnect(#[from] io::Error),      // 属性宏  
7     #[error("the data for key `{0}` is not available")] // 属性宏  
8     Redaction(String),  
9     #[error("invalid header (expected {expected:?}, found {found:?})")] // 属性宏  
10    InvalidHeader {  
11        expected: String,  
12        found: String,  
13    },  
14    #[error("unknown data store error")] // 属性宏  
15    Unknown,  
16 }
```

[复制代码](#)

上面示例中的 `Debug` 和 `Error` 就是派生宏。 `Debug` 宏由 `std` 提供， `Error` 由 `thiserror` crate 提供。使用的时候，我们先使用 `use` 将宏导入到当前 crate 的 scope。然后在 `#[derive()]` 中使用它。

 复制代码

```
1 #[derive(Error, Debug)]
```

它会作用在下面紧跟着的 `enum DataStoreError` 这个类型上面，对这个类型进行某种代码变换操作。最后的实现效果就是帮助我们实现了 `std::error::Error` 这个 trait，以及对应的 `Debug`、`Display` 等 trait。在我们看来，写上这一句话，就好像 Rust 自动帮我们完成了那些实现，给我们省了很多力气。

想看它到底在 `enum DataStoreError` 类型上施加了哪些魔法，只需要运行 `cargo expand` 展开它就可以了。你可以自己动手做一下实验。

在这个 `enum DataStoreError` 里的变体上，我们还可以看到另外两个宏 `#[error()]` 和 `#[from]`，它们不是派生宏，而是过程宏中的属性宏。

认识过程宏之属性宏

Rust 编译器提供了一些属性（Attributes），属性是施加在模块、crate 或 item 上的元数据。这些元数据可用于很多地方。

代码条件编译；

设置 crate 名字，版本号和类型（是二进制程序还是库）；

禁止代码提示；

开启编译器特性（宏、全局导入等）；

链接到外部库；

标记函数为单元测试；

标记函数为性能评测的一部分；

属性宏；

.....

要把属性施加到整个 crate，语法是在 crate 入口，比如在 lib.rs 或 main.rs 的第一行写上 `#![crate_attribute]`。

如果只想把属性施加到某个模块或者 item 上，就把 `!` 去掉。

 复制代码

```
1 #[item_attribute]
```

属性上面还可以携带参数，可以写成下面几种形式：

`#[attribute = "value"]`

`#[attribute(key = "value")]`

`#[attribute(value)]`

我们来看一下具体的例子。

 复制代码

```
1 // 声明这个 crate 为 lib，是全局性的属性
2 #![crate_type = "lib"]
3
4 // 声明下面这个函数为单元测试函数，这个属性只作用在test_foo()函数上
5 #[test]
6 fn test_foo() {
7     /* ... */
8 }
9
10 // 条件编译属性，这块深入下去细节非常多
11 #[cfg(target_os = "linux")]
12 mod bar {
13     /* ... */
14 }
15
16 // 正常来说，Rust中的类型名需要是Int8T这种写法，下面这个示例让编译器不要发警告
```



```
17 #[allow(non_camel_case_types)]
18 type int8_t = i8;
19
20 // 作用在整个函数内部，对未使用的变量不要报警
21 fn some_unused_variables() {
22     #![allow(unused_variables)]
23
24     let x = ();
25     let y = ();
26     let z = ();
27 }
```

如果你开始写原型代码的时候，经常出现变量或函数未使用的情况，Rust 编译器会提示一堆，有时看着心烦，你可以在 crate 入口文件头写上两行内容。

[复制代码](#)

```
1 #![allow(unused_variables)]
2 #![allow(dead_code)]
3
4 fn foo() {
5     let a = 10;
6     let b = 20;
7 }
```

它会压制 Rust 编译器，让 Rust 编译器“放放水”。你可以试着把前面两行注释掉，看看 Rust 编译器会提示什么。但是对于一个严肃的项目来说，应该尽可能地消除这些“未使用”的警告，所以项目写到一定程度，就要把这种全局属性去掉。

Rust 中有非常丰富的属性，它们给 Rust 编译器提供了非常强大的配置能力。要掌握它们，需要花费大量的时间，你可以看一下 [Attributes](#) 相关资料。这也印证了 Rust 的适用面极其广泛。即使一个熟练的程序员，也没必要掌握全部，只需要了解其中的常见属性就可以。然后在遇到一些具体的场景时，再深入研究那些配置。

而如果我们要定义自己的“属性”，就需要通过属性宏来实现。这属于偏高级而且用得比较少的内容，需要的时候再去学习也来得及。具体的实现可以参考我给出的链接。

<https://earthly.dev/blog/rust-macros/>

比如著名的 Web 开发框架 Rocket，就使用属性宏来配置 URL mapping。

 复制代码

```
1  #[macro_use] extern crate rocket;
2
3  #[get("/<name>/<age>")] // 属性宏
4  fn hello(name: &str, age: u8) -> String {
5      format!("Hello, {} year old named {}!", age, name)
6  }
7
8  #[launch] // 属性宏
9  fn rocket() -> _ {
10     rocket::build().mount("/hello", routes![hello])
11 }
```

可以看到，有了属性宏的加持，Rust 代码在结构上变得相当紧凑而有美感，跟 Java 的 Spring 框架有点相似了。

认识过程宏之函数宏

在某些特定的情况下，函数宏能更好地表达业务需求，比如写 SQL 语句。我们在 Rust 中撰写 SQL 语句，必须符合 Rust 的语法，因此，一般使用字符串来构造和传递 SQL 语句，比如：

 复制代码

```
1  fn foo() {
2      let sql = "select title, content from article where id='1111111';";
3  }
```

其实就不是那么方便，还容易出错。如果用函数宏来实现的话，可以做到如下效果：

 复制代码

```
1  // 这是伪代码
2  use sql_macros::sqlbuilder;
```

```
3 fn foo() {  
4     sqlbuilder!(select title, content from article where id='1111111');  
5 }  
6
```

这样写，就好像在一个在 SQL 编辑器里面写 SQL 语句一样，非常自然，而不是把 SQL 语句写成字符串的形式了。也就是说，可以在 Rust 代码中写非 Rust 语法的代码，有没有感觉很神奇。当你想自己造一个 DSL（领域特定语言 Domain Specific Language）时，函数宏就可以派上用场了。

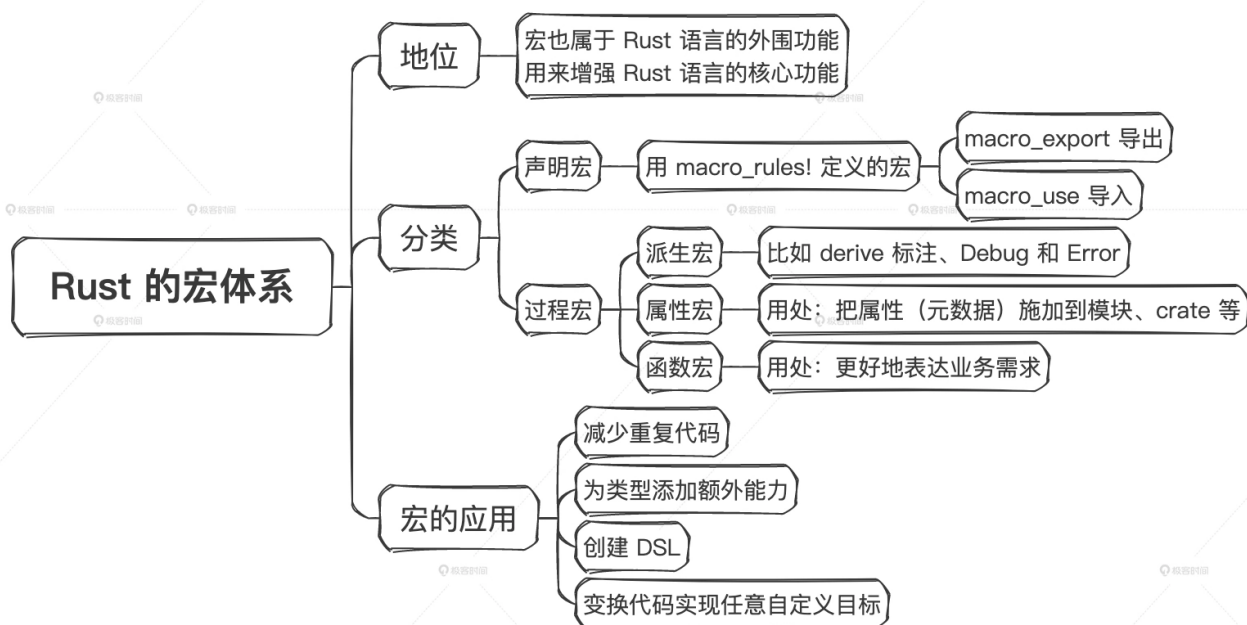
函数宏是过程宏的一种，也使用过程宏的方式来实现，相关信息可参考我给出的 [🔗 链接](#)。目前社区中关于过程宏完整而细致的教程并不多，后面我会出一系列这方面的专题教程。

宏的应用

综上，我们可以看到，利用宏可以做到以下几方面的事情。

1. 减少重复代码。如果有大量的样板代码，可以使用声明宏或派生宏让代码变得更简洁、紧凑。
2. 为类型添加额外能力。这是派生宏和属性宏的强大威力。
3. 创建 DSL。可以自己创建一种新的语言，利用 Rust 编译器在 Rust 中编译运行，而不需要你再去写一个单独的编译器或解释器了。
4. 变换代码实现任意自定义目标。本质上来说，宏就是对代码的变换，并且是在真正的编译阶段之前完成的，因此你可以用宏实现任意天马行空的想法。

小结



宏在 Rust 里无处不在，我们学习的第一步是要认识它们，知道它们的作用，熟悉常见宏的意义和用法。然后要初步掌握写简单的声明宏的写法，这样能有效地提升你精简业务代码的能力。但同时也要注意，使用宏不能过度，宏的缺点是比较难调试，IDE 对它的支持可能也不完美。滥用宏会导致代码难以理解。

过程宏的能力非常强，书写的难度也比较大，我们目前不需要掌握它的写法。当你遇到一个过程宏的时候，你可以先查阅文档，知道它的作用，做到会用。等遇到需要的场景的时候，再去深入钻研。

这节课我们还提到了一个用来展开宏代码的工具 cargo expand，经常使用它，对你学习宏会有很大帮助。

思考题

学完这节课的内容，你可以查阅一下相关资料，说一说 allow、warn、deny、forbid 几个属性的意义和用法。欢迎你把查阅到的内容分享到评论区，如果觉得有收获的话，也欢迎你把这节课的内容分享给其他朋友，我们下节课再见！

全部留言 (4)

最新 精选



Noah•^•

2023-12-01 来自浙江

allow、warn、deny、forbid: 编译器的警告级别

作者回复: 对的



1



Taozi

2023-12-02 来自上海

过程宏本质上是编译器的扩展插件

作者回复: 对的 👍



伯阳

2023-12-01 来自北京

感觉和注解有点像

作者回复: 属性宏和注解有点像



Distance

2023-12-01 来自北京

```
macro_rules! add {  
    // 第一个分支, 匹配两个元素的加法  
    ($a:expr, $b:expr)=>{  
        {  
            $a+$b  
        }  
    }  
}
```

```
};  
// 第二个分支：当只有一个元素时，也应该处理，这是边界情况  
($a:expr)=>{  
    {  
        $a  
    }  
}  
}
```

为什么第二个分支匹配不需要分号呢？ 我看第一个示例只有单个匹配也是分号结尾的

作者回复：非常细心！👍。第二个分支后面也可以加分号的。分号的作用用来间隔多个分支。

共 2 条评论 >

